

CSS @function At-Rule

Complete Guide: Easy English Edition

Master custom functions and advanced CSS programming techniques

Contents

1	Introduction	2
1.1	What is @function?	2
1.2	Why Use @function?	2
2	@function Basics	3
2.1	Basic Syntax	3
2.1.1	Simple Example	3
2.2	Function Parts	3
3	Function Parameters	4
3.1	Single Parameter	4
3.2	Multiple Parameters	4
3.3	Default Parameters	4
3.4	Type Parameters	4
4	Calculations and Operations	6
4.1	Math Operations	6
4.2	Complex Calculations	6
4.3	Conditional Logic	7
5	Real-World Examples	8
5.1	Example 1: Spacing System	8
5.2	Example 2: Typography Scale	8
5.3	Example 3: Color Variations	8
5.4	Example 4: Responsive Values	9
6	Advanced Function Features	10
6.1	Recursive Functions	10
6.2	Functions Calling Functions	10
6.3	Function with Custom Properties	10
7	Comparison with Alternatives	12
7.1	@function vs. Mixins	12
7.2	@function vs. CSS Variables	12
8	Browser Support	13
8.1	Current Status	13
8.2	Preprocessor Alternative	13
9	Common Patterns	14
9.1	Pattern 1: Design Tokens Function	14
9.2	Pattern 2: Responsive Scale	14
9.3	Pattern 3: Color System	14

10 Best Practices	16
10.1 Naming Conventions	16
10.2 Keeping Functions Simple	16
10.3 Documentation	16
11 Common Mistakes	17
11.1 Mistake 1: Missing Return Statement	17
11.2 Mistake 2: Wrong Parameter Types	17
11.3 Mistake 3: Infinite Recursion	17
12 Quick Reference	19
12.1 Function Syntax	19
12.2 Common Function Types	19
12.3 Key Points	19

1 Introduction

The CSS `@function` at-rule is an advanced feature that lets you define reusable functions in CSS. Think of it like writing your own CSS functions with parameters and return values!

1.1 What is `@function`?

The `@function` at-rule allows you to:

- **Define Functions:** Create reusable CSS functions
- **Use Parameters:** Pass values to your functions
- **Return Values:** Functions compute and return results
- **Reduce Duplication:** DRY principle for CSS

1.2 Why Use `@function`?

- **Code Reusability:** Write once, use many times
- **Less Repetition:** Eliminate duplicate calculations
- **Better Maintainability:** Central place to update logic
- **Complex Calculations:** Advanced math and transformations
- **Cleaner Code:** More semantic and readable CSS

2 @function Basics

2.1 Basic Syntax

```
@function function-name(parameter1, parameter2) {  
    return value-or-calculation;  
}
```

2.1.1 Simple Example

```
@function double(number) {  
    return number * 2;  
}  
  
.element {  
    width: double(100px); /* Returns 200px */  
    height: double(50px); /* Returns 100px */  
}
```

2.2 Function Parts

Part	Purpose	Example
@function	Keyword to define function	-
function-name	Name of your function	calculateSize
(parameters)	Input values	(base, multiplier)
return	Output value	return base * multiplier;

3 Function Parameters

3.1 Single Parameter

```
@function square(side) {  
  return side * side;  
}  
  
.box {  
  width: square(10px);    /* 100px */  
  height: square(10px);  /* 100px */  
}
```

3.2 Multiple Parameters

```
@function calculate-area(width, height) {  
  return width * height;  
}  
  
.rectangle {  
  /* Area of 300 x 200 = 60000px2 */  
  --area: calculate-area(300px, 200px);  
}
```

3.3 Default Parameters

```
@function padding(size, multiplier: 1) {  
  return size * multiplier;  
}  
  
.small-padding {  
  padding: padding(8px);    /* 8px */  
}  
  
.large-padding {  
  padding: padding(8px, 3); /* 24px */  
}
```

3.4 Type Parameters

```
@function convert-color(color: <color>) {  
  return color;  
}
```

```
@function scale-size(size: <length>) {  
  return size * 2;  
}  
  
.element {  
  background: convert-color(blue);  
  width: scale-size(100px);  
}
```


4 Calculations and Operations

4.1 Math Operations

```
@function add(a, b) {  
  return a + b;  
}  
  
@function subtract(a, b) {  
  return a - b;  
}  
  
@function multiply(a, b) {  
  return a * b;  
}  
  
@function divide(a, b) {  
  return a / b;  
}  
  
@function modulo(a, b) {  
  return a % b;  
}
```

4.2 Complex Calculations

```
@function fibonacci(n) {  
  if (n <= 1) {  
    return n;  
  } else {  
    return fibonacci(n - 1) + fibonacci(n - 2);  
  }  
}  
  
@function calculate-golden-ratio(size) {  
  return size * 1.618;  
}  
  
@function responsive-font(base, ratio) {  
  return base * ratio;  
}  
  
h1 {  
  font-size: responsive-font(16px, 2);  
}
```

4.3 Conditional Logic

```
@function get-color(brightness) {  
  if (brightness > 50) {  
    return white;  
  } else {  
    return black;  
  }  
}  
  
@function calculate-margin(screen-size) {  
  if (screen-size > 1200px) {  
    return 40px;  
  } else if (screen-size > 768px) {  
    return 20px;  
  } else {  
    return 10px;  
  }  
}
```

5 Real-World Examples

5.1 Example 1: Spacing System

```
@function space(multiplier) {
  return multiplier * 8px;
}

:root {
  --space-1: space(1); /* 8px */
  --space-2: space(2); /* 16px */
  --space-3: space(3); /* 24px */
  --space-4: space(4); /* 32px */
  --space-5: space(5); /* 40px */
}

.element {
  padding: var(--space-3);
  margin: var(--space-2);
}
```

5.2 Example 2: Typography Scale

```
@function type-scale(step, base-size: 16px, ratio: 1.25) {
  return base-size * pow(ratio, step);
}

:root {
  --font-size-xs: type-scale(-2);
  --font-size-sm: type-scale(-1);
  --font-size-base: type-scale(0);
  --font-size-lg: type-scale(1);
  --font-size-xl: type-scale(2);
  --font-size-xxl: type-scale(3);
}

body {
  font-size: var(--font-size-base);
}

h1 {
  font-size: var(--font-size-xxl);
}
```

5.3 Example 3: Color Variations

```

@function lighten(color, amount) {
  return color + amount;
}

@function darken(color, amount) {
  return color - amount;
}

@function get-button-color(state) {
  if (state == 'hover') {
    return darken(#2196F3, 10%);
  } else if (state == 'active') {
    return darken(#2196F3, 20%);
  } else {
    return #2196F3;
  }
}

.button {
  background: #2196F3;
}

.button:hover {
  background: get-button-color('hover');
}

.button:active {
  background: get-button-color('active');
}

```

5.4 Example 4: Responsive Values

```

@function responsive(mobile, tablet, desktop) {
  if (viewport-width < 768px) {
    return mobile;
  } else if (viewport-width < 1200px) {
    return tablet;
  } else {
    return desktop;
  }
}

.container {
  padding: responsive(12px, 24px, 40px);
  font-size: responsive(14px, 16px, 18px);
}

```

6 Advanced Function Features

6.1 Recursive Functions

```
@function power(base, exponent) {  
  if (exponent == 0) {  
    return 1;  
  } else {  
    return base * power(base, exponent - 1);  
  }  
}  
  
.scaled {  
  width: power(2, 4)px; /* 16px */  
}
```

6.2 Functions Calling Functions

```
@function base-size(multiplier) {  
  return multiplier * 8px;  
}  
  
@function padding-size(level) {  
  return base-size(level) + 4px;  
}  
  
.box {  
  padding: padding-size(2); /* 20px */  
}
```

6.3 Function with Custom Properties

```
:root {  
  --base-unit: 8px;  
  --color-primary: #2196F3;  
}  
  
@function unit(multiplier) {  
  return multiplier * var(--base-unit);  
}  
  
@function color-variant(base-color, modifier) {  
  return base-color; /* Simplified */  
}
```

```
.element {  
  padding: unit(3);  
  background: color-variant(var(--color-primary), 'dark');  
}
```

7 Comparison with Alternatives

7.1 @function vs. Mixins

Mixins (Less/Sass):

```
@mixin padding($size) {  
  padding: $size * 2;  
}  
  
.element {  
  @include padding(10px);  
}
```

@function (CSS):

```
@function padding($size) {  
  return $size * 2;  
}  
  
.element {  
  padding: padding(10px);  
}
```

7.2 @function vs. CSS Variables

Variables:

```
:root {  
  --padding-small: 8px;  
  --padding-medium: 16px;  
}  
  
.element {  
  padding: var(--padding-medium);  
}
```

Functions:

```
@function padding(multiplier) {  
  return multiplier * 8px;  
}  
  
.element {  
  padding: padding(2); /* 16px */  
}
```

8 Browser Support

8.1 Current Status

Browser	Support
Chrome	Experimental (behind flag)
Firefox	Under development
Safari	Under consideration
Edge	Experimental (behind flag)

Note: @function is still in early development. Use preprocessors (Sass, Less) for production!

8.2 Preprocessor Alternative

Use Sass functions for production code:

```
@function scale(base, multiplier) {  
  @return base * multiplier;  
}  
  
.element {  
  width: scale(10px, 5); /* 50px */  
}
```


9 Common Patterns

9.1 Pattern 1: Design Tokens Function

```
@function token(category, level) {
  if (category == 'spacing') {
    return level * 8px;
  } else if (category == 'font-size') {
    return level * 2px + 12px;
  } else if (category == 'border-radius') {
    return level * 2px;
  }
}

.element {
  padding: token('spacing', 3);
  font-size: token('font-size', 2);
  border-radius: token('border-radius', 2);
}
```

9.2 Pattern 2: Responsive Scale

```
@function fluid-size(min-size, max-size, min-vw, max-vw) {
  return calc(
    min-size +
    (max-size - min-size) *
    (100vw - min-vw) /
    (max-vw - min-vw)
  );
}

h1 {
  font-size: fluid-size(24px, 48px, 320px, 1280px);
}
```

9.3 Pattern 3: Color System

```
@function shade(color, amount) {
  return darken(color, amount);
}

@function tint(color, amount) {
  return lighten(color, amount);
}
```

```
.button {  
  background: #2196F3;  
}  
  
.button:hover {  
  background: shade(#2196F3, 10%);  
}  
  
.button-light {  
  background: tint(#2196F3, 20%);  
}
```

10 Best Practices

10.1 Naming Conventions

```
/* Good: Descriptive names */
@function calculate-aspect-ratio(width, height) { ... }
@function get-breakpoint-size(breakpoint) { ... }
@function convert-pixels-to-rem(pixels) { ... }

/* Avoid: Unclear names */
@function calc(a, b) { ... }
@function get(x) { ... }
@function fn() { ... }
```

10.2 Keeping Functions Simple

```
/* Good: Single responsibility */
@function double(value) {
  return value * 2;
}

/* Avoid: Multiple responsibilities */
@function process-value(value, operation, format) {
  /* Too complex */
}
```

10.3 Documentation

```
/* Function: Calculate responsive spacing
 * Parameters:
 *   - base: Base spacing unit (default: 8px)
 *   - multiplier: Multiplication factor
 * Returns: Calculated spacing value
 */
@function space(multiplier, base: 8px) {
  return base * multiplier;
}
```

11 Common Mistakes

11.1 Mistake 1: Missing Return Statement

DON'T:

```
@function double(value) {  
  value * 2;  /* No return! */  
}
```

DO:

```
@function double(value) {  
  return value * 2;  
}
```

11.2 Mistake 2: Wrong Parameter Types

DON'T:

```
@function add(a, b) {  
  return a + b;  
}  
  
/* Mixing types */  
width: add(10px, 'hello');
```

DO:

```
@function add(a: <length>, b: <length>) {  
  return a + b;  
}  
  
width: add(10px, 5px);  /* 15px */
```

11.3 Mistake 3: Infinite Recursion

DON'T:

```
@function loop(n) {  
  return loop(n);  /* Infinite loop! */  
}
```

DO:

```
@function countdown(n) {  
  if (n == 0) {  
    return 0;  
  } else {
```

```
    return countdown(n - 1);  
  }  
}
```

12 Quick Reference

12.1 Function Syntax

```
/* Basic function */
@function name(param1, param2) {
  return computation;
}

/* With default parameters */
@function name(param1, param2: default-value) {
  return computation;
}

/* With type specifications */
@function name(param1: <type>, param2: <type>) {
  return computation;
}

/* Conditional logic */
@function name(param) {
  if (condition) {
    return value1;
  } else {
    return value2;
  }
}
```

12.2 Common Function Types

Type	Purpose
Calculation	Math operations on values
Conditional	Return different values based on conditions
Transform	Change input format or type
Recursive	Call itself with modified parameters
Composition	Call other functions

12.3 Key Points

- Define functions with `@function` keyword
- Use `return` statement for output
- Support parameters with default values

- Can include conditional logic
- Support recursion with base cases
- Still experimental - use Sass for production
- Progressive enhancement keeps styles working

Conclusion

CSS `@function` at-rule represents the future of programmatic styling in CSS. While still experimental, it brings the power of functional programming to CSS, making code more maintainable and DRY.

Key Takeaways:

- `@function` enables code reusability in CSS
- Create functions with parameters and return values
- Support complex calculations and logic
- Use for design systems and token generation
- Progressive enhancement with fallbacks
- Monitor browser support as it evolves

For production use, consider Sass functions as they have full browser support today. Experiment with `@function` as it becomes available!